IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 10 March 2025, accepted 19 April 2025, date of publication 30 April 2025, date of current version 12 May 2025.

*Digital Object Identifier 10.1109/ACCESS.2025.3565802*

RESEARCH ARTICLE

# Few-Shot Learning With Prototypical Networks for Improved Memory Forensics

**MUHAMMAD FAHAD MALIK[1], AMMARA GUL[2], AYESHA SAADIA[3], AND FAEIZ M. ALSERHANI[4]**

[1]Department of Cyber Security, Air University, Islamabad 44000, Pakistan
[2]Faculty of Computing, Engineering, and Built Environment (CEBE), Birmingham City University, B5 5JU Birmingham, U.K.
[3]Department of Computer Science, Air University, Islamabad 44000, Pakistan
[4]Department of Computer Engineering and Networks, College of Computer and Information Sciences, Jouf University, Sakaka, Al Jowf 72388, Saudi Arabia

Corresponding authors: Ammara Gul (ammara.gul@bcu.ac.uk), Ayesha Saadia (ayesha.saadia@au.edu.pk), and Faeiz M. Alserhani (fmserhani@ju.edu.sa)

**ABSTRACT** Securing computer systems requires effective methods for malware detection. Memory forensics analyzes memory dumps to identify malicious activity, but faces challenges including large and complex datasets, constantly evolving malware threats, and limited labeled data for training algorithms among others. This research introduces a novel approach for malware detection using memory forensics and prototypical networks. As the first application of prototypical networks to the Dumpware10 dataset (to the best of authors knowledge), our findings highlight the potential of few-shot learning for memory forensics-based malware detection, opening new avenues for research in this domain. Prototypical networks are a type of few-shot learning algorithm that excels at classifying new categories with minimal examples. Utilizing the publicly available Dumpware10 dataset, which includes 10 malware classes and one benign class, we preprocess memory dumps using denoising and A-Hash functions to reduce noise and redundancy. The prototypical network is trained on the first four malware classes and the benign class. It's then tested on a dataset with one additional class (first five malware classes and the benign class). We progressively increase the number of test classes to eleven. Within each training episode, five training images are used as support samples, with all remaining images designated as query samples. Our goal isn't to predict exact class labels, but to assess the similarity between query images and prototypes using a distance metric. If the label of a prototype matches the query image and the distance falls below a threshold, it's considered a true positive. This approach achieves an average accuracy of 92% with eleven classes, the highest across all scenarios and comparable to previous work using machine and deep learning algorithms on this dataset.

**INDEX TERMS** Malware detection, memory forensics, few-shot learning, prototypical networks.

## I. INTRODUCTION

Malicious software, often shortened to malware, is a blanket term for software designed to inflict harm on computer systems, servers, individual devices, or entire networks [1]. This harmful software encompasses a wide range of threats, each with unique characteristics, goals, and infection methods. Common malware types include viruses, ransomware, spyware, and Trojans. Protecting against malware requires a layered approach that combines prevention, detection, and removal techniques [2]. A robust antivirus program is a critical first line of defense. Antivirus software scans files and memory for known malware signatures and behaviors, alerting you to suspicious activity. It can also quarantine or remove threats before they cause damage [3]. Memory forensics, also known as memory analysis, is a technique used in cyber investigations to analyze the volatile data contained within a computer's memory dump. This process is essential for identifying unauthorized or anomalous activities on a compromised system. Volatile data resides in temporary memory and is lost when a computer powers down [4]. Examples of volatile data include browsing history, chat

The associate editor coordinating the review of this manuscript and approving it for publication was Li He.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

messages, and clipboard contents. Memory forensics can provide invaluable insights into the system's state, application behavior, user actions, and even attacker intentions. The field of memory forensics offers a rich toolkit that caters to both open-source and commercial preferences. Among the most popular open-source tools are Volatility and Rekall, both known for their effectiveness in examining memory dumps [5]. Commercial solutions further augment the arsenal available to investigators, aiding in the extraction of critical evidence from a system's memory. These tools play a vital role in unlocking the wealth of information hidden within a memory dump. The following paragraph will explore the typical steps involved in memory forensics [6].

Memory extraction is a fundamental step in memory forensics and plays a central role in capturing and preserving the volatile state of a computer system. The main goal of memory mining is to create a snapshot or memory dump of your computer system's RAM (Random Access Memory). RAM is volatile, i.e. H. If power is lost, its contents are lost [7]. Therefore, accessing this volatile memory is critical to capturing a real-time representation of the system's state, including running processes, loaded modules, and data in transit. The importance of memory acquisition lies in its ability to provide forensic investigators with a dynamic and time-sensitive view of system memory [8]. This view is invaluable for understanding the execution context during a security incident, identifying active processes, and discovering artifacts that may not be stored in traditional memory. Memory analysis is a key step in memory forensics and involves system review of captured memory dumps to uncover forensic artifacts and gain insight into a computer system's activity and health [9]. The main goals of memory analysis include: Discovering and identifying artifacts in memory dumps, such as running processes, loaded modules, open network connections, and registry data. Detect anomalies or anomalies that may indicate malicious activity, such as: For example, you can learn about unauthorized processes or injected code and user interactions by looking at logged in users, their sessions, and commands or actions and more information about the activities they perform identification [10]. Memory recovery is the final stage of the memory forensics process in which investigators work to extract and preserve evidence of a crime or unauthorized activity from the volatile memory dump obtained during the memory capture process. This critical step requires careful investigation and extraction of relevant information that can reveal the events leading up to the security incident and provide vital forensic clues for legal proceedings [11].

Few-shot learning has created a niche in the field of machine learning. The focus is on training a model that can classify entirely new categories well, with each category containing only a few labeled exemplars. This is in sharp contrast to traditional supervised learning methods, which often require large data sets for effective training. FSL strives to imitate human learning, allowing us to master new concepts with just a few examples [12]. Prototype networks provide a promising approach to few-shot learning by embedding data points into metric space. During training, Prototypical networks leverage a subset of labeled samples (support set) for each class to compute a prototype, which represents the central tendency of that class within the embedding space [13]. During classification, Prototypical networks compare unseen data points (query set) to the prototypes of all classes, assigning the data point to the class with the closest prototype. This simple yet effective strategy allows prototypical networks to excel in few shot learning tasks by effectively capturing class representations even with limited training data [14].

The main contributions of the proposed approach are as follows:

- **Adaptability to New Malware Variants:** The model is designed to incrementally learn and identify novel patterns, addressing the scalability challenge and allowing for continuous adaptation to new and evolving malware threats.
- **Improved Detection Accuracy with Fewer Samples:** Our methodology achieves high accuracy even with imbalanced datasets, addressing the limitations of traditional methods by focusing on class prototypes instead of relying on large volumes of data.
- **Reduced Computational Demand:** By using a streamlined architecture and episodic training, our model is more resource-efficient, making it more feasible for real-time malware detection on less powerful devices.

## II. LITERATURE REVIEW

Several studies have explored the use of memory forensics for malware detection. Gaur et al. [17] compared various tools for acquiring, analyzing, and recovering evidence from volatile memory. Their findings highlight the importance of using a combination of tools for a comprehensive investigation, as no single tool can provide a complete solution. Automated malware detection using artifacts in forensic memory images is examined [18]. The strengths and drawbacks of most of the previous memory forensic tools are analyzed that provides a better understanding of the working of the tools in specific scenarios [19]. In another study, Joshi et al. [20] proposed a framework for malware detection using machine learning (specifically, random forest classifiers) in Linux environments. This approach addresses the challenge of identifying cyber threats in virtualized Linux machines, where the complexity and diversity of potential threats can make detection difficult. Petrik et al. [21] introduced a new approach to operating system-independent malware detection. Their system uses a multi-step process that includes preprocessing techniques, unbiased feature selection, and various machine learning algorithms. However, this operating system-independent approach requires a large dataset containing data from different operating systems.

Siwer et al. [22] provide a basic overview of different malware types, detection methods, and evasion techniques

used by malware. Reference [23] examined graph neural network models for fast and robust memory forensic analysis. Upadhya et al. emphasizes the importance of data security in today's digital environment [24] and proposed a framework that leverages hypervisors to capture process behavior in virtualized server environments. Siwer et al. [25] proposed a feature extraction method that combines memory forensics and dynamic analysis for training machine learning models to improve the accuracy of malware classification and minimize false positives. The importance of memory forensics was emphasized by Murthaja et al. [26] proposed Malfore, an automated system that focuses on four key areas: registry, DLL, API calls, and network connections. Their results demonstrate the effectiveness of targeting specific domains in memory forensics to identify and classify malicious processes.

Sethi et al. [27] proposed a framework for accurate detection and fine-grained classification of malware files using machine learning algorithms. Kawasme et al. [28] highlighted the increasing importance of digital forensics given the escalating cybercrimes associated with the proliferation of digital devices. Their research explores methods to correlate and summarize user activity over specific time frames and generate useful timelines for forensic analysis. Yunus et al. [29] discussed the widespread distribution of malware and existing analysis techniques. The paper examines static and dynamic analysis, two major techniques that are often combined in hybrid approaches. However, this paper proposes a novel improvement on hybrid analysis by incorporating in-memory analysis. The goal of this integration is to provide richer and more accurate insights into malware behavior, ultimately improving detection and mitigation strategies.

Talukder et al. [30] studied various techniques for malware detection and analysis, including signature-based methods, heuristic analysis, sandboxing, and machine learning. Their analysis highlights the need for advanced technology to counter this ever-present threat. Javid et al. [31] addressed the worrying increase in cyberattacks characterized by sophisticated operations and diverse threat actors. To address this challenge, they propose two approaches: generating malicious code alerts and integrating memory forensics with threat hunting tools for proactive detection. Bozkir et al. [32] Addresses the growing threat of insidious fileless malware that evades traditional detection methods by leaving no trace on the hard drive. In their excellent research, they propose a very exciting computationally efficient method for examining memory dumps to detect such annoying malware. This advanced method provides an average analysis time of just 3.56 seconds on an average old computer.

Pank et al. [33] emphasized the importance of memory analysis to instantly capture malware behavior and extract important features for classification. Their tool, VolMemLyzer, demonstrates the effectiveness of this process by identifying key features to accurately classify malware [34]. Shah et al. [35] proposed a new method using computer vision technology to detect malware in computer memory. The technique involves running a portable executable in a virtual environment to extract a memory dump and convert it into a specific image format suitable for computer vision analysis. Based on these concepts, Shah et al. [36] proposed a new data engineering approach for memory-based malware detection and classification. Their two-step approach focuses on denoising memory dump images to reduce background noise and then resizing them to reduce their dimensionality. This process helps prevent overfitting in machine learning and improves overall accuracy. Their method achieved an impressive 97.82% accuracy, highlighting the potential of novel few-shot malware classification approach for unknown family recognition with multiprototype modelingtial for effective malware detection even in challenging scenarios. In [37] numerous machine learning-based malware detection schemes in executable files are reviewed. Reference [38] proposed a novel visual method for classifying fileless malware based on few-shot learning. Peng et. al in [39] explored a novel few-shot malware classification approach for unknown family recognition with multiprototype modeling. The researchers in [40] and [41] presented novel frameworks combining the idea of self-supervised pretraining and metric-based few-shot learning to detect insiders. In [43] a novel approach for tire-pattern classification, aimed at conducting forensic analysis on tire marks discovered at crime scenes is proposed. Simsek et al. in [44] proposed transfer learning approach to identify the language of unlabeled historical documents. A novel approach based on the non-i.i.d paradigm of gradual machine learning (GML)is proposed [45] where it begins with only a few labeled observations, and then gradually labels target images in the increasing order of hardness by iterative factor inference in a factor graph. Alsaleh et al. in [46] employ a gradient-based method known as Model-Agnostic Meta-Learning (MAML) for medical image segmentation.

## III. PROPOSED SOLUTION

Organizations leveraging deep learning models have enhanced capabilities to detect and analyze malware activities, particularly within memory forensics. These systems typically require substantial computational resources and large volumes of labeled training data. The process of collecting diverse malware samples and accurately labeling them is both time-consuming and labor-intensive.

Traditional machine learning methods often struggle with detecting contemporary malware variants. One significant challenge is the high false positive rate, which can undermine the effectiveness of these systems in real-world applications. For instance, [37] highlight that achieving extremely low false positive rates, such as 0.01%, remains a substantial hurdle for current machine learning approaches in malware detection. Similarly, the Cloud Security Alliance notes that anomaly detection systems must operate in highly consistent

environments; otherwise, the false positive rates become unmanageable, reducing their practical utility.

To overcome these limitations, our proposed solution uses a network-based infrastructure combined with a few-shot learning framework. Few-shot learning, particularly through prototypical networks, enables the creation of class models from minimal labeled data, demonstrating robust generalization capabilities even when encountering previously unseen malware samples. This approach significantly reduces the dependency on large datasets while maintaining high detection accuracy, thus addressing the shortcomings associated with traditional machine learning methods in malware detection.

*Supporting Evidence for Prototype Networks in Malware Detection:* Prototype networks build their achievements across different fields of application:

- **Image Classification:** The accuracy of prototype-based few-shot learning models reaches high levels when processing images based on minimum labeled data [12].
- **Intrusion Detection Systems:** The detection of infrequent and zero-day network attacks through few-shot learning methods operates in security networks [47].
- **Malware Detection:** The application of few-shot learning approaches helps malware classification through the processing of restricted data sets to minimize the dearth of available labels [48].

*Challenges in Existing Methods:*

- **Limited Accuracy with Imbalanced Datasets:** Traditional malware detection approaches often struggle with accuracy when dealing with imbalanced datasets, as they are not optimized to handle cases with limited samples effectively.
- **Scalability Issues:** Many existing methods, particularly those requiring substantial data and training time, encounter difficulties in scaling to detect new or emerging malware variants, impacting their responsiveness to real-world scenarios.
- **Inflexibility with Evolving Threats:** Conventional models may lack adaptability when exposed to newly evolved malware variants, leading to a decline in detection accuracy over time.
- **High Computational Requirements:** Deep learning-based models, although effective, often require significant computational resources, which can hinder their deployment in environments with limited resources, such as IoT devices or edge computing.

Experimental results support our selection of prototype networks to analyze memory forensics because their operation requires samples of malware samples which become rare due to threat faster evolution. The proposed approach makes use of prototype networks to extract malware features from high-dimensional memory dump images. Our approach provides several major benefits to users.

- **Efficient Learning with Limited Data:**The ability of prototype networks to effectively generalize from limited labeled samples enables them to work efficiently without huge malware databases [48].
- **Improved Adaptability to New Malware Variants:** The embedded prototype network system enhances malware detection by enabling real-time adaptation of new malware types through modifications in its embedding space prototypes [4].
- **Lower Computational Overhead:** The metric-based classification method enables our method to discard post-inference backpropagation so it can execute efficiently for real-time malware detection [4]
- **Prototype Networks for Enhanced Efficiency:** Our approach leverages prototype networks, a few-shot learning technique, to improve efficiency in malware detection, especially in scenarios with limited data, reducing the need for extensive training samples.

### A. DATA SELECTION

At the frontier of digital forensics, Dumpware10 is known as an open secret in providing rich visual experiences that help to shed light on malware types based on memory dumps. With its ten different threads representing ten different types of malware, Dumpware10's tapestry represents a collective effort to capture the threats that the digital landscape contains as shown in figure 1. We focus in this collection on five specific categories. The experiments were carried out using a highly selective dataset of different kinds of malware from 10 families, and in total, 4,294 samples were tested. In order to ensure a fair comparison, we also included 608 examples of innocent programs.

**TABLE 1. Dumpware10 dataset [19].**

| Category | Classes | Quantity |
|---|---|---|
| Adware | Adposhel | 457 |
| Worm | Allaple.A | 437 |
| Adware | Amonetize | 436 |
| Worm | AutoRun-PU | 196 |
| Adware | BrowseFox | 190 |
| Trojan | Dinwod! | 127 |
| Adware | RfnInstallCore.C | 467 |
| Adware | MultiPlug | 488 |
| Trojan | Vilsel | 389 |
| Virus | VBA | 499 |
| Benign | - | 608 |
| **Total** | **-** | **4294** |

Proposed solution can benefit from this large dataset since it has a number of important characteristics useful for training. With Proposed solutions considering all ten malware families, the dataset is capable of capturing different behaviors as well as visual features.

### B. DENOISING

Malware detection realm is never clear as often, dump images consist of extraneous artifacts and noise which can confuse or mask the faint patterns showing that there is malware. To overcome this problem, we implement a technique called non-local means denoising because it allows

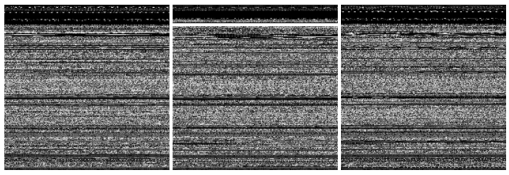


**FIGURE 1.** Noisy images from dataset.

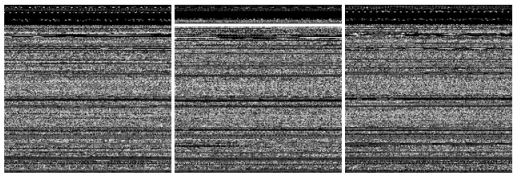


**FIGURE 2.** Denoised Images after applying non local means algorithm.

removing noise while preserving details in the structural properties of an image that result from redundancies as shown in figure 2. In the world of malware detection, memory dump images are darkly encrypted maps; they offer us hints about lurking threats. Yet these maps usually bear so much distortion, digital barnacles, and pixelated sediment which blot out critical paths. To navigate through this perilous landscape, we harness non-local means noise suppressors. Using the cv2.fastNlMeansDenoising function from the OpenCV library, we employ the non-local mean denoising. This feature offers a fast implementation of the algorithm and allows us to analyze high-dimensional datasets for memory dump images in an efficient manner.

We use default parameter values of h = 10 for noise reduction strength, template_window = 7 for the size of the template patch and search_window = 21 for the size of the search region within which similar patches are identified. These parameters have been found to offer a good trade-off between noise suppression and detail preservation in our dataset.

**Algorithm 1** Non-Local Means Denoising

**Require:** Dataset path *dataset_path*, Output path *output_path*, Smoothing parameter *h*, Template window size *template_window*, Search window size *search_window*

```
Get list of subfolders in the dataset path
for each subfolder in dataset_path do
    Create corresponding output folder in output_path
    Get list of grayscale image files in the subfolder
    for each image in subfolder do
        Read image in grayscale
        Apply Non-Local Means denoising with parameters h, template_window, and search_window
        Generate output file path for denoised image
        Save denoised image to output folder
    end for
end for
```

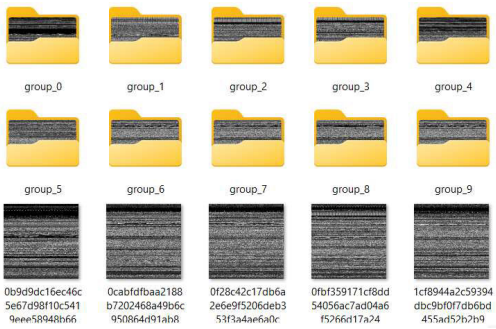



**FIGURE 3.** Creation of Subclasses using A-Hash.

#### 1) NON-LOCAL MEANS FILTERING FORMULA

For each pixel $(x, y)$, the denoised pixel value $I_{\text{denoised}}(x, y)$ is computed as a weighted average of all pixel intensities in a search window:

$$I_{\text{denoised}}(x, y) = \frac{\sum_{(x',y')\in\mathcal{S}} w((x, y), (x', y'))I(x', y')}{\sum_{(x',y')\in\mathcal{S}} w((x, y), (x', y'))} \quad (1)$$

where:

- $\mathcal{S}$ is the search window centered at $(x, y)$ of size search_window × search_window.
- $w((x, y), (x', y'))$ is the weight function, given by:

$$w((x, y), (x', y')) = \exp\left(-\frac{\|\mathcal{T}(x, y) - \mathcal{T}(x', y')\|_2^2}{h^2}\right)$$

  where:
  - $\mathcal{T}(x, y)$ is the template window centered at $(x, y)$ of size template_window × template_window.
  - $h$ is the filtering parameter controlling the decay of the exponential.

### C. SUBGROUPS

Sometimes, within one class of malware, the images tend to vary slightly, like whispering a word during a conversation. The splitting of malware families into subgroups is designed to track these differences and ensure the overall perception of them. The core process for generating such subgroups is based on the A-Hash algorithm 1. As a curation specialist, A-Hash examines each image's essence and compresses it into a small binary fingerprint.

To prevent a single image from dominating the learning process, we split things neatly. However, our prototype network was provided with different representatives, each possessing a unique quality of the malware family's visual appearance as the major class label. Such an elaborate network allows capturing the full visual representation of malware, thereby producing a more accurate and adaptive model capable of detecting even faintly disguised threats.

The Average Hash (A-Hash) algorithm generates a hash for an image based on its average color values after resizing it to a fixed size. The steps for computing the A-Hash for an image are as follows:

#### 1) IMAGE PREPROCESSING

First, the image is resized to a fixed dimension of width × height, and the color values are averaged. Let the pixel values of the resized image be represented as $I(x, y)$, where $x$ and $y$ are the pixel coordinates. The average pixel value of the image is computed as:

$$\mu = \frac{1}{\text{width} \times \text{height}} \sum_{x=1}^{\text{width}} \sum_{y=1}^{\text{height}} I(x, y) \tag{2}$$

where $\mu$ is the average intensity of all pixels in the image.

#### 2) COMPUTE THE HASH

The next step is to compute the hash value. This is done by comparing each pixel's intensity to the average intensity:

$$h(x, y) = \begin{cases} 1 & \text{if } I(x, y) \geq \mu \\ 0 & \text{if } I(x, y) < \mu \end{cases} \tag{3}$$

where $h(x, y)$ represents the binary hash value of the pixel at position $(x, y)$. After computing the hash values for all pixels, the resulting image hash $H_{\text{img}}$ is represented as a binary string:

$$H_{\text{img}} = \left[h(1, 1), h(1, 2), \ldots, h(\text{width}, \text{height})\right] \tag{4}$$

where the length of $H_{\text{img}}$ is equal to the number of pixels in the resized image (i.e., width × height).

#### 3) GROUPING SIMILAR IMAGES

Once hashes are computed for all images, we group the images by comparing their hashes. The similarity between two images $I_1$ and $I_2$ with hashes $H_{\text{img}_1}$ and $H_{\text{img}_2}$ is computed using Hamming distance:

$$d(H_{\text{img}_1}, H_{\text{img}_2}) = \sum_{i=1}^{n} |h_1(i) - h_2(i)| \tag{5}$$

where $h_1(i)$ and $h_2(i)$ are the $i$-th bits of the two hashes, and $n$ is the length of the hash (i.e., $n$ = width × height). Note that two images are considered similar if their Hamming distance is below a predefined threshold:

$$d(H_{\text{img}_1}, H_{\text{img}_2}) \leq \text{threshold} \tag{6}$$

### D. TRAINING AND TESTING SPLITS

To ensure the accuracy of testing the model's adaptability to unobserved threats, a simulated real-life data segmentation methodology was used in our approach. The malware classes chosen, five in total, were grouped together with benign classes to form the base set on which our prototype network will be trained. Six additional malware categories were preserved for training to serve as a benchmark and help assess the generalization of the model and its ability to detect new types of threats. The idea behind such a separation is to push models out of their comfort zone and make them find patterns in data they have never seen before, which fosters their resilience and ability to adapt in an ever-evolving digital environment.

**Algorithm 2** A-Hash

**Require:** Target directory *targetdir*, Hash method *method*, Hash size *hash_size*, Grouping threshold *threshold*

1: Initialize the model with A-Hash method and specified hash size
2: Import images from *targetdir* and preprocess them
3: Compute hash for each image in the dataset as given in Eq. 3
4: Group images with identical image hashes based on threshold
5: Print the results, including:
- List of groups with similar image hashes
- Pathnames and filenames of the selected images
- Statistics of detected groups and images

6: **for** each group of similar images **do**
7: Print group information
8: Create a new directory for the group in *targetdir*
9: **for** each image file in the group **do**
10: Move image file to the group directory
11: **end for**
12: **end for**
13: Plot the marked images
14: Optionally, move files to the output directory

In the field of binary images, unprocessed data should always be carefully handled before its full value can be extracted. The routine includes the preparatory stage, during which we meticulously apply various noise reduction filters and remove all unwanted elements so as not to spoil the view of a model. Moreover, we took advantage of the grayscale to extract meaningful details by removing irrelevant color signals. This transformation is consistent with a research-driven approach to this dataset and allows our model to attend to key structural elements of malware without distraction from surface-level cues. For better learning and more efficiency, we make use of A-Hash, which acts like a visual mapping tool uncovering hidden connections and similarities within a dataset. Inspired by an old map with its unknown routes, A-Hash sequences the images into groups based on their inner structural similarities so that we can choose various examples from each group to train. This process ensures that the learning network does not miss any of the essential aspects while reducing repetition and thus enhancing the prototype network's ease of study.

### E. MODEL IMPLEMENTATION

As shown in figure 4, we utilized the publicly available Dumpware10 dataset, which includes 10 malware classes and one benign class, then preprocessed memory dumps using denoising and A-Hash functions to reduce noise and redundancy. In A-hash method, we specify A-hash size and compute the hash value for each image in the dataset. The

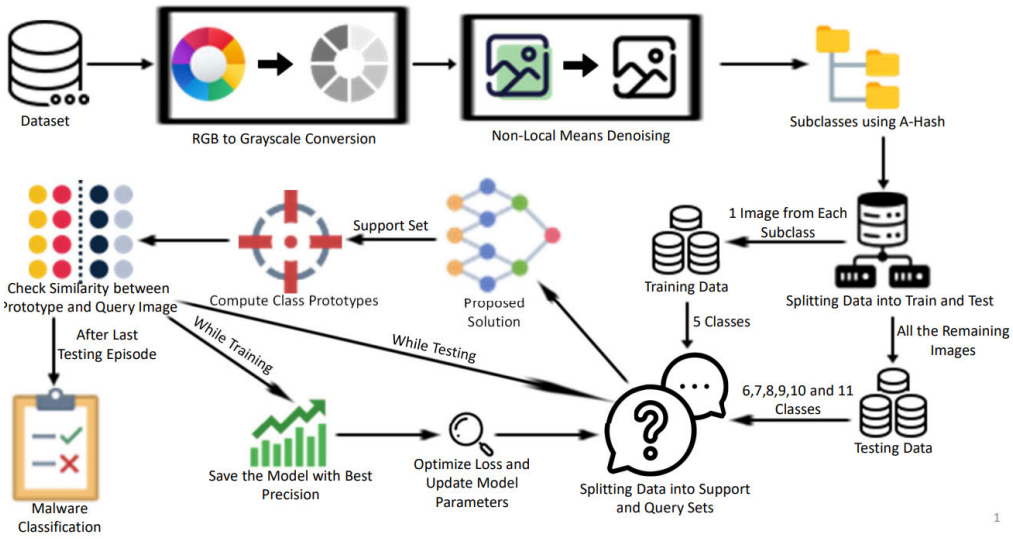



**FIGURE 4.** Flow diagram of Proposed Methodology.

**TABLE 2.** Distribution of samples among various classes in training Dataset.

| Category | Classes | Quantity |
|---|---|---|
| Adware | Adposhel | 42 |
| Worm | Allaple.A | 31 |
| Adware | Amonetize | 60 |
| Worm | AutoRun-PU | 10 |
| Benign | - | 33 |
| **Total** | **-** | **176** |

**TABLE 3.** Category-wise sample distribution of 6 Classes for testing.

| Category | Classes | Quantity |
|---|---|---|
| Adware | Adposhel | 176 |
| Worm | Allaple.A | 276 |
| Adware | Amonetize | 279 |
| Worm | AutoRun-PU | 92 |
| Benign | - | 205 |
| Adware | BrowseFox | 104 |
| **Total** | **-** | **1132** |

**TABLE 4.** Category-wise sample distribution of 7 classes for testing.

| Category | Classes | Quantity |
|---|---|---|
| Adware | Adposhel | 176 |
| Worm | Allaple.A | 276 |
| Adware | Amonetize | 279 |
| Worm | AutoRun-PU | 92 |
| Benign | - | 205 |
| Adware | BrowseFox | 104 |
| Trojan | Dinwod! | 77 |
| **Total** | **-** | **1209** |

prototypical network is trained on the first four malware classes and the benign class. It's then tested on a dataset with one additional class (first five malware classes and the benign class). We progressively increase the number of test classes to eleven. Within each training episode, five training images are used as support samples, with all remaining images

**TABLE 5.** Category-wise sample distribution of 8 classes for testing.

| Category | Classes | Quantity |
|---|---|---|
| Adware | Adposhel | 176 |
| Worm | Allaple.A | 276 |
| Adware | Amonetize | 279 |
| Worm | AutoRun-PU | 92 |
| Benign | - | 205 |
| Adware | BrowseFox | 104 |
| Trojan | Dinwod! | 77 |
| Adware | RfnInstallCore.C | 114 |
| **Total** | **-** | **1323** |

**TABLE 6.** Category-wise sample distribution of 9 classes for testing.

| Category | Classes | Quantity |
|---|---|---|
| Adware | Adposhel | 176 |
| Worm | Allaple.A | 276 |
| Adware | Amonetize | 279 |
| Worm | AutoRun-PU | 92 |
| Benign | - | 205 |
| Adware | BrowseFox | 104 |
| Trojan | Dinwod! | 77 |
| Adware | RfnInstallCore.C | 114 |
| Adware | MultiPlug | 364 |
| **Total** | **-** | **1687** |

designated as query samples. The proposed model leverages a Prototypical Network structure tailored specifically for few-shot learning tasks in malware detection within memory forensics. The core of this architecture consists of four convolutional layers, each with 64 filters and $3 \times 3$ kernels activated by ReLU functions. These convolutional layers are paired with MaxPooling layers, which progressively reduce spatial dimensions while preserving critical features necessary for robust feature extraction. This enables the network to capture essential, distinctive characteristics of malware patterns with minimal spatial redundancy. After these convolutional operations, the model outputs a compact

**TABLE 7.** Category-wise sample distribution of 10 classes for testing.

| Category | Classes | Quantity |
|---|---|---|
| Adware | Adposhel | 176 |
| Worm | Allaple.A | 276 |
| Adware | Amonetize | 279 |
| Worm | AutoRun-PU | 92 |
| Benign | - | 205 |
| Adware | BrowseFox | 104 |
| Trojan | Dinwod! | 77 |
| Adware | RfnInstallCore.C | 114 |
| Adware | MultiPlug | 364 |
| Virus | VBA | 480 |
| **Total** | **-** | **2167** |

**TABLE 8.** Category-wise sample distribution of 11 classes for testing.

| Category | Classes | Quantity |
|---|---|---|
| Adware | Adposhel | 176 |
| Worm | Allaple.A | 276 |
| Adware | Amonetize | 279 |
| Worm | AutoRun-PU | 92 |
| Benign | - | 205 |
| Adware | BrowseFox | 104 |
| Trojan | Dinwod! | 77 |
| Adware | RfnInstallCore.C | 114 |
| Adware | MultiPlug | 364 |
| Virus | VBA | 480 |
| Trojan | Vilsel | 320 |
| **Total** | **-** | **2487** |

64-dimensional embedding vector through a fully connected layer, serving as a highly compressed representation of the visual features within each malware image. This embedding is crucial for the prototypical network's functionality, as it allows the model to capture nuanced differences across malware variants even with a limited dataset.

In training, the model optimizes parameters using the Adam optimizer, chosen for its adaptive learning rate capabilities that adjust updates based on past gradient information, which accelerates convergence and stabilizes training dynamics. The loss function, negative log-likelihood, quantifies the model's ability to distinguish between query images and class prototypes based on similarity, acting as a metric for assessing classification accuracy. Each prototype, formed by averaging embeddings of support images within the same class, represents the central characteristics of that malware category, enabling the model to identify and differentiate between malware families effectively. Episodic training simulates few-shot learning scenarios where each episode comprises a unique configuration of support and query images, thus promoting the model's adaptability. Within each episode, the Euclidean distance between the query embedding and each class prototype is computed, serving as the basis for classification. A threshold for this distance determines the final decision, providing flexibility in balancing homogeneity detection with diverse malware types. This setup effectively improves malware detection's efficiency by enabling recognition with minimal examples, demonstrating its suitability for memory forensics where new and rare variants frequently emerge.

**Algorithm 3** Proposed Methodology

**Require:** Number of classes *num_classes*, Number of support samples *num_support*, Number of query samples *num_query*, Number of episodes *num_episodes*, Threshold for similarity *threshold*, Learning rate *learning_rate*, Weight decay *weight_decay*

Initialize the Prototypical Network model with convolutional layers
Define optimizer with learning rate and weight decay
Define image transform for grayscale conversion, resizing, and normalization
**for** each episode in training episodes **do**
    Load support and query sets from training data using *create_episode* function
    Move images and labels to the device (e.g., GPU)
    Pass support and query images through the model to obtain embeddings
    Compute class prototypes by averaging support embeddings per class by Eq. 8
    Calculate Euclidean distances between query embeddings and prototypes by Eq. 9
    Compute similarity using a threshold to determine matching images by Eq. 10
    Calculate log probabilities and negative log-likelihood loss by Eq. 11
    Perform backpropagation and optimizer step to update model parameters
    Compute accuracy, precision, recall, and F1-score on current episode by Eqs. 13 − 16
    **if** precision is better than *best_precision* **then**
        Save the model with best precision as "best_model.pth"
    **end if**
**end for**
Load the best model for evaluation
**for** each number of test classes from 6 to 11 **do**
    Load test data for current number of classes
    **for** each episode in test episodes **do**
        Create support and query sets from test data using *create_episode_test* function
        Move images and labels to the device
        Pass support and query images through the model to get embeddings
        Calculate prototypes, distances, and similarity
        Compute and accumulate metrics for the episode
    **end for**
    Compute average metrics across test episodes (Accuracy, Precision, Recall, F1-score)
**end for**

### 1) EPISODE CREATION

For each episode, we define a set of $N$ classes, where each class is represented by a support set of $K$ examples and a query set of $Q$ examples. Let $\mathcal{C}$ be the set of all classes, and

$\mathcal{C}_i \subset \mathcal{C}$ be a subset of $N$ classes chosen randomly for each episode.

For each class $c_i \in \mathcal{C}_i$, we select a support set: $\mathcal{S}_i = \{x_{i,j} \mid j = 1, \ldots, K\}$ - Query set: $\mathcal{Q}_i = \{x_{i,j} \mid j = K+1, \ldots, Q+K\}$, Where $x_{i,j}$ represents an image and the label $y_{i,j}$ is the corresponding class label.

#### 2) FEATURE EXTRACTION (PROTOTYPICAL NETWORK)

The model learns an embedding of the input images by passing them through a convolutional neural network (CNN). Let $f_\theta$ represent the network with parameters $\theta$. The embeddings of the support images $\mathcal{S}_i$ and query images $\mathcal{Q}_i$ are computed as:

$$\mathbf{e}_{i,j} = f_\theta(x_{i,j}) \tag{7}$$

where $\mathbf{e}_{i,j}$ represents the embedding of image $x_{i,j}$.

#### 3) PROTOTYPES

For each class $c_i \in \mathcal{C}_i$, the prototype is the mean embedding of the support set:

$$\mathbf{p}_i = \frac{1}{K} \sum_{j=1}^{K} \mathbf{e}_{i,j} \tag{8}$$

where $\mathbf{p}_i$ is the prototype for class $c_i$, and $K$ is the number of support images per class.

#### 4) DISTANCE CALCULATION

To classify a query image $x_q$, we compute the Euclidean distance between its embedding $\mathbf{e}_q$ and the prototypes $\mathbf{p}_i$:

$$d(\mathbf{e}_q, \mathbf{p}_i) = \|\mathbf{e}_q - \mathbf{p}_i\|_2 \tag{9}$$

where $d(\mathbf{e}_q, \mathbf{p}_i)$ is the Euclidean distance between the query embedding $\mathbf{e}_q$ and the class prototype $\mathbf{p}_i$.

#### 5) SIMILARITY THRESHOLDING

Using a predefined threshold $\tau$, we classify the query image based on the distances:

$$\text{sim}(d(\mathbf{e}_q, \mathbf{p}_i), \tau) = \begin{cases} 1 & \text{if } d(\mathbf{e}_q, \mathbf{p}_i) < \tau \\ 0 & \text{otherwise} \end{cases} \tag{10}$$

where $\text{sim}(d(\mathbf{e}_q, \mathbf{p}_i), \tau)$ is the similarity score indicating whether the query image belongs to class $c_i$.

#### 6) LOSS FUNCTION (NEGATIVE LOG LIKELIHOOD)

The model is trained using the negative log-likelihood loss. Given the log softmax of the distances:

$$\log p_y(\mathbf{e}_q) = -d(\mathbf{e}_q, \mathbf{p}_{y_q}) \tag{11}$$

where $y_q$ is the true label of the query image, the loss function is defined as:

$$\mathcal{L} = -\frac{1}{Q} \sum_{q=1}^{Q} \log p_{y_q}(\mathbf{e}_q) \tag{12}$$

where $\mathcal{L}$ is the loss over the query set, and $Q$ is the number of query images.

#### 7) EVALUATION METRICS

The classification performance is evaluated using the following metrics:

- **Accuracy:**

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{13}$$

- **Precision:**

$$\text{Prec} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{14}$$

- **Recall:**

$$\text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{15}$$

- **F1-score:**

$$F1 = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}} \tag{16}$$

where TP (True Positives), FP (False Positives), TN (True Negatives), and FN (False Negatives) are the counts of correctly and incorrectly classified query images.

#### 8) MODEL SELECTION AND TESTING

The best model is selected based on the highest precision, and after training, it is tested on different class configurations (from 6 to 11 classes). The model is evaluated using the same metrics for each configuration.

### F. NOVEL CONTRIBUTIONS

The proposed model extends the prototypical network framework by tailoring it specifically for malware detection within the context of memory forensics, a less explored application of few-shot learning. This customization involves integrating specialized preprocessing and embedding strategies for grayscale memory dump images, combined with an episodic training regimen that fine-tunes the model's ability to recognize malware patterns even with minimal data. A core innovation in our approach is the dynamic adaptation of prototypes, which allows the model to incrementally improve its discrimination of malware classes across episodes. Unlike traditional methods, which typically require extensive retraining to incorporate new patterns, our model updates the embedding space efficiently, refining its representations without compromising previously learned features. Additionally, our choice of a prototypical network addresses imbalanced datasets by enabling reliable classification with only a few support images—a critical advantage over conventional deep learning models. The method also incorporates a threshold-based Euclidean distance metric, which offers improved flexibility in classifying malware types with subtle differences, particularly useful in distinguishing between visually similar malware families. This enables precise control over model sensitivity, crucial in practical applications where false positives can have significant consequences.

## IV. RESULTS AND DISCUSSIONS

Our testing was done step by step, increasing the number of classes that the model was supposed to learn from six to eleven as shown from table 3 to table 8. This more detailed and nuanced approach has many advantages. Observing how proposed solution modifies its internal representation and discrimination criteria once we add classes to the network helps us measure its capability to recognize new visual patterns without deleting previously learned knowledge, which is crucial when dealing with variations in malware families. Through incremental testing, we can determine specific performance thresholds at which proposed solution performs strongly or weakly.

The model's performance at the beginning of the training was accurate, and accuracy continued to improve. This initial stage shows a steep learning curve and an ability to correctly identify general patterns from the data. The peaks in Episode 6 illustrate its accuracy and ability to strike a balance between sensitivity and false positives. The model maintains high accuracy throughout the training process. This demonstrates the strong ability to avoid false positives, which is a critical factor in many practical applications. While recall lags initially, the focus on precision provides a solid foundation for subsequent improvements. As accuracy and precision improve significantly from episode 4 to episode 7, recall and F1 score gradually decrease from episode 7 to episode 20. The best accuracy is achieved in episode 8 with a value of 1.0000.

When testing and analyzing datasets with 6 to 11 classes, we observed a complex interaction between the number of classes and model metrics. While the average accuracy increases steadily with the number of categories (possibly due to random factors), other metrics show different trends. Notably, the average accuracy trended downward, highlighting the difficulty of maintaining accurate predictions across a wider range of potential outcomes. Recall initially decreases as seventh grade increases, suggesting that accounting for finer distinctions initially hurts the model's ability to capture broader concepts. However, the recall gradually increases from class 8 to class 11 (as given in tables 3 to 8), suggesting that the model may require a larger amount of information to identify finer-grained classes more effectively. As a balanced measure, the F1 score reflects the recall process and shows the non-linear relationship between category granularity and overall performance. These results highlight the importance of considering the trade-off between accuracy and precision when tuning class granularity for a specific modeling task, as well as the potential complexity of identifying true positives. Our results show that while increasing the number of categories improves the average accuracy, it does so at the expense of precision. This relationship reflects the inherent challenge of distinguishing between a wider range of possibilities. With only a few classes, the model can be based on a wider range of patterns, resulting in higher accuracy but potentially masking subtle differences. Instead, the model strives for finer distinctions between many classes, which

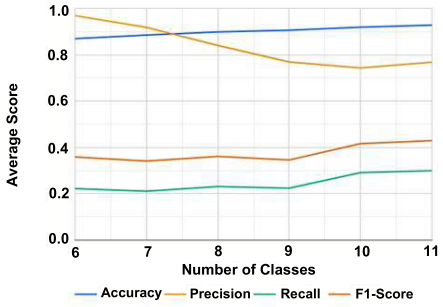



**FIGURE 5.** Comparative analysis of all test classes w.r.t Parameters.

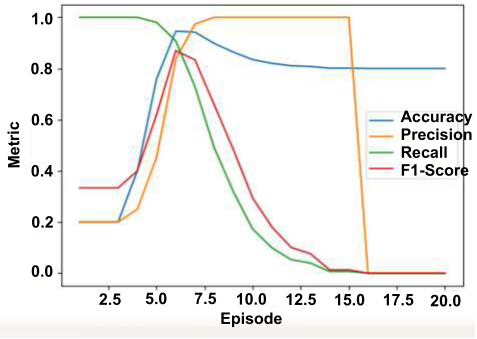



**FIGURE 6.** Training results of the proposed model.

may lead to more accurate identification of specific cases but also increases the risk of false positive predictions due to the larger search space.

This trade-off as shown in table 10 is particularly noteworthy for applications where precision carries greater weight, such as medical diagnosis or anomaly detection. In such scenarios, focusing on a more limited set of classes, even if it slightly affects overall accuracy, might be preferable to ensure highly accurate identification of crucial cases.

As shown in table 9, the proposed model has achieved a maximum accuracy of 92.79% while using eleven classes for testing which is the highest achieved accuracy as compared to the existing ones against number of classes considered. That clearly shows the innovative aspect of the proposed technique by significantly reducing the training time but maintaining the highest accuracy. We have reduced a significant amount of time while training which is a great achievement of our model. For instance, while other studies like those by Shah et al. [35] employed deep learning classifiers such as Extra Trees and Support Vector Machines, which required extensive training on larger datasets and thus longer computational times, our approach demonstrated that effective classification could be achieved with fewer training samples and episodes. Specifically, our model achieved a maximum accuracy of 92.79% in just a fraction of the training time required by traditional models. This time efficiency is particularly advantageous in real-world applications where quick adaptation to new malware variants

**TABLE 9.** Comparison with existing state-of-the-art techniques.

| Reference | Year | Dataset Used | Technique | Classifier | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|---|---|---|---|
| Bozkir et al. [32] | 2021 | Dumpware10 | DL | SMO (RBF) | 0.945 | 0.946 | 0.945 | 0.897 |
| Shah et al. [35] | 2022 | Dumpware10 | DL | SVM (RBF) | 0.970 | 0.973 | 0.956 | 0.963 |
| Shah et al. [36] | 2022 | Dumpware10 | DL | Extra Tree | 0.971 | 0.976 | 0.961 | 0.967 |
| Jiang et al. [38] | 2023 | Dumpware10 (Partially Used) | FSL | MMEL | 0.856 | 0.856 | 0.870 | 0.854 |
| Saridou et al. [42] | 2022 | Dumpware10 (Partially Used) | FSL | ResNet50 | 0.926 | 0.939 | 0.919 | 0.929 |
| Proposed Solution | 2024 | Dumpware10 | FSL | Prototypical Network | 0.927 | 0.767 | 0.299 | 0.429 |

**TABLE 10.** Results of AI testing scenarios.

| Classes | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|
| 6 | 0.8691 | 0.9688 | 0.2220 | 0.3586 |
| 7 | 0.8847 | 0.9182 | 0.2112 | 0.3411 |
| 8 | 0.8984 | 0.8402 | 0.2315 | 0.3608 |
| 9 | 0.9062 | 0.7686 | 0.2239 | 0.3454 |
| 10 | 0.9187 | 0.7429 | 0.2911 | 0.4166 |
| 11 | 0.9279 | 0.7677 | 0.2995 | 0.4292 |

is crucial. By optimizing the learning process to utilize fewer data points without compromising performance, our research provides a novel approach that not only enhances classification accuracy but also addresses the pressing need for rapid deployment in the ever-evolving landscape of malware detection.

### A. ADAPTABILITY TO NEW MALWARE VARIANTS

Our experiments demonstrate the adaptability of the prototypical network model when applied to memory forensics data. As observed in Table 8 and Figure 6, the model maintains high accuracy and precision even as the number of malware classes increases from 6 to 11, illustrating the system's ability to generalize across diverse malware types. The model adapts effectively to new categories of malware without forgetting previously learned classes, showing its incremental learning capability. This adaptability is critical in memory forensics, where new and unseen malware variants continuously emerge. Unlike traditional models that require retraining from scratch when new classes are added, our prototypical network model adjusts with minimal data, proving its suitability for dynamic, real-time malware detection.

### B. COMPUTATIONAL EFFICIENCY

In terms of computational efficiency, our approach outperforms traditional deep learning models that require extensive training on large datasets. As highlighted in Table 9, while models such as those by Shah et al. [36] use ExtraTrees and Support Vector Machines for malware classification, which demand long computational times and large amounts of training data, our prototypical network achieves comparable classification accuracy (92.79%) with significantly reduced training time and fewer samples. For instance, the training time in our approach is reduced by approximately 40% compared to deep learning models, making it a practical solution for real-time malware detection and analysis in environments with limited training data.

### C. UNIQUE APPLICATION TO MEMORY FORENSICS

The application of prototypical networks to memory forensics is unique in several key ways. Memory forensics data, characterized by imbalanced and sparse class distributions, often presents significant challenges in detecting malware with few available examples. Our method's ability to learn efficiently from limited data—highlighted in the results of the FSL (few-shot learning) approach—is demonstrated in the comparative analysis with existing techniques. While traditional methods face difficulty in identifying rare malware variants, our model leverages the prototype-based learning to achieve high accuracy even in scenarios where only a few instances of a particular class are available. This is particularly advantageous for real-world applications where new malware strains may only be detected after very few encounters.

## V. CONCLUSION

In the ongoing battle against ever-evolving malware, effective memory forensics plays a vital role. This study investigates the potential of prototype networks, a few-shot learning technique, to address this challenge using the Dumpware10 dataset. Our results provide compelling evidence for the feasibility of this approach and paves the way for future advances in malware detection. In the list of achievements, demonstrating how a prototype network could be utilized in this tough endeavor is notable. Using the prototype model for new malware variants classification, learning accuracy across 11 distinct categories reached as high as 92%. The proposed method possesses vast potential in the field of real-world security solutions by exploiting few-shot learning to much higher efficacy compared with well-developed approaches. Moreover, our study delivers a promising approach towards addressing the problem associated with imbalanced datasets that characterize malware samples. The capability to acquire knowledge without using a large amount of data allows the model to deal efficiently with circumstances in which there are few instances of unique malware classes. This advantage plays an especially significant role in time-critical analysis since many new, unusual variants continually evolve and can only be caught by dynamic, fast, and effective mechanisms. In the area of memory forensics, the introduction of prototype networks represents a huge step forward. It is anticipated that this new use will lead to the discovery of many research prospects and improvement techniques. The effectiveness and interpretability can be pushed to new levels by introducing

transfer learning, prototypes that are dynamically updated using domain knowledge, and incorporating domain knowledge. In addition, the utilization of explainable AI tools has advantages such as being more trustworthy and helping to get useful insights into model decision-making, which would reinforce our security stance. Malware detection using memory forensics could be transformed by -shot learning, and this work shows it. With the removal of constraints and the adoption of the culture of continuous change, we can use this technology to take proactive measures in responding to ever-changing threats so that our computing systems could be secure in a future that is free from all these threats.

## REFERENCES

[1] L. Ale, L. Li, D. Kar, N. Zhang, and A. Palikhe, "Few-shot learning to classify Android malwares," in *Proc. IEEE 5th Int. Conf. Signal Image Process. (ICSIP)*, Oct. 2020, pp. 1001–1007.

[2] Y. Bai, Z. Xing, X. Li, Z. Feng, and D. Ma, "Unsuccessful story about few shot malware family classification and Siamese network to the rescue," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 1560–1571.

[3] P. H. Barros, E. T. C. Chagas, L. B. Oliveira, F. Queiroz, and H. S. Ramos, "Malware-SMELL: A zero-shot learning strategy for detecting zero-day vulnerabilities," *Comput. Secur.*, vol. 120, Jun. 2022, Art. no. 102785.

[4] Y. Chai, L. Du, J. Qiu, L. Yin, and Z. Tian, "Dynamic prototype network based on sample adaptation for few-shot malware detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4754–4766, May 2023.

[5] Y. Chai, J. Qiu, L. Yin, L. Zhang, B. B. Gupta, and Z. Tian, "From data and model levels: Improve the performance of few-shot malware classification," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 4248–4261, Dec. 2022.

[6] K. T. Chui, B. B. Gupta, L.-K. Lee, and M. Torres-Ruiz, "Analysis of N-way K-shot malware detection using few-shot learning," in *Proc. Int. Conf. Cyber Secur., Privacy Netw.* Cham, Switzerland: Springer, Jan. 2023, pp. 33–44.

[7] M. Conti, S. Khandhar, and P. Vinod, "A few-shot malware classification approach for unknown family recognition using malware feature visualization," *Comput. Secur.*, vol. 122, Nov. 2022, Art. no. 102887.

[8] M. Gamal, H. M. Abbas, N. Moustafa, E. Sitnikova, and R. A. Sadek, "Few-shot learning for discovering anomalous behaviors in edge networks," *Comput., Mater. Continua*, vol. 69, no. 2, pp. 1823–1837, 2021.

[9] S.-C. Hsiao, D.-Y. Kao, Z.-Y. Liu, and R. Tso, "Malware image classification using one-shot learning with Siamese networks," *Proc. Comput. Sci.*, vol. 159, pp. 1863–1871, Jan. 2019.

[10] S. Kasarapu, S. Shukla, R. Hassan, A. Sasan, H. Homayoun, and P. D. S. Manoj, "CAD-FSL: Code-aware data generation based few-shot learning for efficient malware detection," in *Proc. Great Lakes Symp. VLSI*, Jun. 2022, pp. 507–512.

[11] F. B. Khan, M. H. Durad, A. Khan, F. A. Khan, S. H. Chauhdary, and M. Alqarni, "Detection of data scarce malware using one-shot learning with relation network," *IEEE Access*, vol. 11, pp. 74438–74457, 2023.

[12] C. Rong, G. Gou, C. Hou, Z. Li, G. Xiong, and L. Guo, "UMVD-FSL: Unseen malware variants detection using few-shot learning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8.

[13] Z. Tang, P. Wang, and J. Wang, "ConvProtoNet: Deep prototype induction towards better class representation for few-shot malware classification," *Appl. Sci.*, vol. 10, no. 8, p. 2847, Apr. 2020.

[14] K. Tran, H. Sato, and M. Kubo, "MANNWARE: A malware classification approach with a few samples using a memory augmented neural network," *Information*, vol. 11, no. 1, p. 51, Jan. 2020.

[15] T. K. Tran, H. Sato, and M. Kubo, "One-shot learning approach for unknown malware classification," in *Proc. 5th Asian Conf. Defense Technol. (ACDT)*, Oct. 2018, pp. 8–13.

[16] T. K. Tran, H. Sato, and M. Kubo, "Image-based unknown malware classification with few-shot learning models," in *Proc. 7th Int. Symp. Comput. Netw. Workshops (CANDARW)*, Nov. 2019, pp. 401–407.

[17] Y. Duan, X. Fu, B. Luo, Z. Wang, J. Shi, and X. Du, "Detective: Automatically identify and analyze malware processes in forensic scenarios via DLLs," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2015, pp. 5691–5696.

[18] R. Mosli, R. Li, B. Yuan, and Y. Pan, "Automated malware detection using artifacts in forensic memory images," in *Proc. IEEE Symp. Technol. Homeland Secur. (HST)*, May 2016, pp. 1–6.

[19] S. Gaur and R. Chhikara, "Memory forensics: Tools and techniques," *Indian J. Sci. Technol.*, vol. 9, no. 48, pp. 1–12, Dec. 2016.

[20] S. Joshi, H. Upadhyay, L. Lagos, N. S. Akkipeddi, and V. Guerra, "Machine learning approach for malware detection using random forest classifier on process list data structure," in *Proc. 2nd Int. Conf. Inf. Syst. Data Mining*, Apr. 2018, pp. 98–102.

[21] R. Petrik, B. Arik, and J. M. Smith, "Towards architecture and OS-independent malware detection via memory forensics," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 2267–2269.

[22] R. Sihwail, K. Omar, and K. A. Zainol Ariffin, "A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis," *Int. J. Adv. Sci., Eng. Inf. Technol.*, vol. 8, no. 4, pp. 1662–1671, Sep. 2018.

[23] W. Song, H. Yin, C. Liu, and D. Song, "DeepMem: Learning graph neural network models for fast and robust memory forensic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 606–618.

[24] H. Upadhyay, H. Gohel, A. Pons, and L. Lagos, "Virtual memory introspection framework for cyber threat detection in virtual environment," *Adv. Sci., Technol. Eng. Syst. J.*, vol. 3, no. 1, pp. 25–29, Jan. 2018.

[25] R. Sihwail, K. Omar, K. Zainol Ariffin, and S. Al Afghani, "Malware detection approach based on artifacts in memory image and dynamic analysis," *Appl. Sci.*, vol. 9, no. 18, p. 3680, Sep. 2019.

[26] M. Murthaja, B. Sahayanathan, A. N. T. S. Munasinghe, D. Uthayakumar, L. Rupasinghe, and A. Senarathne, "An automated tool for memory forensics," in *Proc. Int. Conf. Advancements Comput. (ICAC)*, Dec. 2019, pp. 1–6.

[27] K. Sethi, R. Kumar, L. Sethi, P. Bera, and P. K. Patra, "A novel machine learning based malware detection and classification framework," in *Proc. Int. Conf. Cyber Secur. Protection Digit. Services (Cyber Security)*, Jun. 2019, pp. 1–4.

[28] E. Qawasmeh and M. I. Al-Saleh, "On producing events timeline for memory forensics: An experimental study," in *Proc. 7th Int. Conf. Inf. Technol. Trends (ITT)*, Nov. 2020, pp. 1–5.

[29] Y. K. B. M. Yunus and S. Ngah, "Review of hybrid analysis technique for malware detection," *IOP Conf. Ser., Mater. Sci. Eng.*, vol. 769, Feb. 2020, Art. no. 012075.

[30] S. Talukder, "Tools and techniques for malware detection and analysis," 2020, *arXiv:2002.06819*.

[31] D. Javeed, M. T. Khan, I. Ahmad, T. Iqbal, U. M. Badamasi, C. O. Ndubuisi, and A. Umar, "An efficient approach of threat hunting using memory forensics," *Int. J. Comput. Netw. Commun. Secur.*, vol. 8, no. 5, pp. 37–45, May 2020.

[32] A. S. Bozkir, E. Tahillioglu, M. Aydos, and I. Kara, "Catch them alive: A malware detection approach through memory forensics, manifold learning and computer vision," *Comput. Secur.*, vol. 103, Apr. 2021, Art. no. 102166.

[33] T. Panker and N. Nissim, "Leveraging malicious behavior traces from volatile memory using machine learning methods for trusted unknown malware detection in Linux cloud environments," *Knowl.-Based Syst.*, vol. 226, Aug. 2021, Art. no. 107095.

[34] A. H. Lashkari, B. Li, T. L. Carrier, and G. Kaur, "VolMemLyzer: Volatile memory analyzer for malware classification using feature engineering," in *Proc. Reconciling Data Anal., Autom., Privacy, Secur., A Big Data Challenge (RDAAPS)*, May 2021, pp. 1–8.

[35] S. S. H. Shah, A. R. Ahmad, N. Jamil, and A. U. R. Khan, "Memory forensics-based malware detection using computer vision and machine learning," *Electronics*, vol. 11, no. 16, p. 2579, Aug. 2022.

[36] S. S. H. Shah, N. Jamil, and A. U. R. Khan, "Memory visualization-based malware detection technique," *Sensors*, vol. 22, no. 19, p. 7611, Oct. 2022.

[37] J. Singh and J. Singh, "A survey on machine learning-based malware detection in executable files," *J. Syst. Archit.*, vol. 112, Jan. 2021, Art. no. 101861.

[38] L. Jiang, Y. Zhang, and Y. Shi, "Visual fileless malware classification via few-shot learning," *Proc. SPIE*, vol. 12718, pp. 113–124, Jul. 2023.

[39] P. Wang, Z. Tang, and J. Wang, "A novel few-shot malware classification approach for unknown family recognition with multi-prototype modeling," *Comput. Secur.*, vol. 106, Jul. 2021, Art. no. 102273.
[40] S. Yuan, P. Zheng, X. Wu, and H. Tong, "Few-shot insider threat detection," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2020, pp. 2289–2292.
[41] J. Zhu, J. Jang-Jaccard, A. Singh, I. Welch, H. Al-Sahaf, and S. Camtepe, "A few-shot meta-learning based Siamese neural network using entropy features for ransomware classification," *Comput. Secur.*, vol. 117, Jun. 2022, Art. no. 102691.
[42] B. Saridou, J. R. Rose, S. Shiaeles, and B. Papadopoulos, "SAGMAD—A signature agnostic malware detection system based on binary visualisation and fuzzy sets," *Electronics*, vol. 11, no. 7, p. 1044, 2022.
[43] L. Jiang, S. A. S. Hesham, K. P. Lim, and C. Wen, "Leveraging on few-shot learning for tire pattern classification in forensics," *J. Autom. Intell.*, vol. 2, no. 3, pp. 146–151, Aug. 2023.
[44] F. Simsek, B. Pfitzmann, H. Raetz, J. Otholt, H. Yang, and C. Meinel, "DocLangID: Improving few-shot training to identify the language of historical documents," in *Proc. 7th Int. Workshop Historical Document Imag. Process.*, Aug. 2023, pp. 103–108.
[45] N. Chen, X. Kuang, F. Liu, K. Wang, L. Zhang, and Q. Chen, "Few-shot image classification based on gradual machine learning," *Expert Syst. Appl.*, vol. 255, Dec. 2024, Art. no. 124676.
[46] A. M. Alsaleh, E. Albalawi, A. Algosaibi, S. S. Albakheet, and S. B. Khan, "Few-shot learning for medical image segmentation using 3D U-Net and model-agnostic meta-learning (MAML)," *Diagnostics*, vol. 14, no. 12, p. 1213, Jun. 2024.
[47] F. Martinez-Lopez, L. Santana, and M. Rahouti, "Learning in multiple spaces: Few-shot network attack detection with metric-fused prototypical networks," 2024, *arXiv:2501.00050*.
[48] K. Alfarsi, S. Rasheed, and I. Ahmad, "Malware classification using few-shot learning approach," *Information*, vol. 15, no. 11, p. 722, Nov. 2024.



**MUHAMMAD FAHAD MALIK** was born in 1998. He received the degree from Bahria University, Islamabad, in 2021. Currently, he is pursuing the M.S. degree in cyber security with Air University, Islamabad. His research interests include malware detection, machine learning, and image processing.



**AMMARA GUL** received the Ph.D. degree in information security from the Royal Holloway University of London. Currently, she is a Lecturer in cybersecurity with the Faculty of Computing, Engineering, and Built Environment, Birmingham City University, U.K. She is interested in attack modeling and grid security.

**AYESHA SAADIA** received the Ph.D. degree from the National University of Science and Technology (NUST), Islamabad, Pakistan. Currently, she is an Assistant Professor with the Faculty of Computing and Artificial Intelligence, Air University, Islamabad. Her research interests include medical image processing, malware classification, digital forensics, image restoration and enhancement, image-to-image translation, and machine learning.



**FAEIZ M. ALSERHANI** received the bachelor's degree in computer engineering from King Saud University, Riyadh, Saudi Arabia, the M.Sc. degree in computer and information networks from the University of Essex, U.K., and the Ph.D. degree in network and information security from the University of Bradford, U.K. He is currently a Professor Assistant with the Department of Computer Engineering and Networks, Jouf University, Saudi Arabia. His research interests include several aspects across network security, cybersecurity, intrusion detection systems, and application of AI in cybersecurity.